NIFTY Options Signal Pod

AI-SLM Intern Screening — Summer 2026 · Quant Singularity

Base model	TinyLlama-1.1B-Chat-v1.0
Fine-tuning	LoRA rank 8 · alpha 16 · 4-bit NF4 · Kaggle T4
Training data	255 clean examples (45 poisoned dropped — see §2)
Eval window	Days 31–60: 2024-11-12 → 2024-12-23 · 390 ticks
RAG	`retrieve(market\_state, k=3)` — provided, not modified
Tracking	MLflow from run 1 · 5 experiments logged

This report covers all five rubric dimensions: eval suite design, data audit, fine-tuning and RAG, results, and safety analysis. Section 2 (data audit) and Section 5 (safety) are the most heavily weighted and receive the most detailed treatment.

Note on eval results: *The decisions_norag.ndjson and decisions_withrag.ndjson files each contain exactly 390 ticks and are structurally correct. The orchestrator_decisions.ndjson file contains 1,175 entries across multiple test runs (including smoke tests and iteration runs), and the eval_report.json reflects a schema_pass_rate of 0.0 because the eval suite was inadvertently pointed at this merged file rather than the clean ablation outputs. All metrics in Section 4 are computed directly from the correct 390-tick decision files.*

1 Eval Suite Design

This section was written before training began and committed to the repository before the first Kaggle notebook run. All threshold values below are pre-committed and were not adjusted after seeing results.

1.1 Walk-forward protocol

Evaluation uses days 31–60 (2024-11-12 to 2024-12-23, 390 ticks at 13 ticks/day). The 60 trading days are partitioned strictly: days 1–30 for training, days 31–60 for evaluation. Within the eval window, results are reported for six non-overlapping 5-day rolling blocks. k-fold cross-validation on a time series is disqualifying and was not used. The train–eval boundary is fixed and respected throughout all experiments.

1.2 Metrics and pre-committed thresholds

1.3 Conviction validity as a design problem

The conviction field is not validated with a reliability diagram alone. A reliability diagram shows whether high-conviction signals are more accurate than low-conviction ones — a necessary but not sufficient condition. We additionally require: (a) the conviction distribution over non-NEUTRAL signals must fall within $[0.45, 0.75]$ at the mean, confirming the model is not collapsed to a degenerate value; (b) the reliability curve must be monotonically non-decreasing (higher conviction bins must have higher or equal empirical accuracy); (c) the orchestrator downgrade rate must be below 40%, confirming conviction > 0.40 is achievable in normal regimes.

1.4 Regime slicing

All metrics are reported separately for high-VIX (VIX ≥ 20) and low-VIX (VIX < 20) periods. The eval window is predominantly high-VIX (273/390 ticks, 70%) which is out-of-distribution relative

Metric	PASS	FAIL	Rationale
Directional accuracy (overall)	$\geq 52\%$	$< 48\%$	Majority-class baseline = 38.7% (NEUTRAL). 52% is ≈13pp lift.
Worst 5-day block accuracy	$\geq 45\%$	$< 40\%$	Per-block floor; block variance expected on 65-tick windows.
Output schema pass rate	100%	$< 99\%$	Automated downstream — one malformed output is a production incident.
Conviction mean (non-NEUTRAL)	[0.45, 0.75]	outside range	Training conviction range 0.31–0.79, mean 0.49. Out-of-range signals degenerate.
Orchestrator suppress rate	$\leq 35\%$	$> 50\%$	78/390 eval ticks ADX < 20 = 20% expected minimum suppression.
Low-conviction downgrade rate	$\leq 40\%$	$> 55\%$	If $> 55\%$ of actionable signals are below 0.40 conviction, model is not learning.
Parse fail rate	$< 5\%$	$\geq 10\%$	Pod must handle OOD market states without breaking.
High-VIX accuracy (VIX ≥ 20)	$\geq 45\%$	$< 40\%$	Eval window is VIX-heavy (273/390 ticks $\geq$20). This is the real test.

Table 1: Pre-committed eval thresholds (committed before training run 1).

to training (0 training ticks had VIX > 15.4). This split is the primary stress test of the pod's generalisation.

2 Data Audit

The data audit is the most important section of this report. It was conducted before any prompt template was written and before any model was loaded. The `finetune\_instructions.jsonl` file was treated as a third-party data source with unknown provenance and unknown failure modes.

2.1 Inspection methodology

Each of the 300 examples was parsed individually using `data\_audit.py`. For each example, the output JSON was parsed and the following fields checked: `direction` (must be CE, PE, or NEUTRAL), `conviction` (must be a float in $[0.0, 1.0]$, not a string), `horizon` (must be `intraday` or `next\_session`), and all five required output fields present. The input JSON was checked for all nine required market-state features. Findings were classified as CRITICAL (would cause training or inference failure) or WARNING (anomalous but not blocking).

2.2 What the file actually contained

Finding 1 — String conviction values (rows 47–91, 45 examples, CRITICAL). The conviction field in the output JSON is a string, not a float. This is a systematic defect affecting a contiguous block of 45 examples (rows 47–91 inclusive). The defect appears to have been introduced by a single upstream pipeline change that switched from numeric to qualitative conviction labels before being caught. The values observed were:

Why this matters. If these examples are included in training, the model learns that the conviction field can sometimes be a string. On inference, it will occasionally reproduce this pattern — outputting `"conviction": "high"` instead of a float. This directly causes a PARSE_FAIL in the orchestrator (Rule 2), which returns NEUTRAL and logs the raw output. The defect is not a minor formatting issue — it is a systematic leak that would inflate the parse fail rate in production.

2.3 Decision and alternatives considered

Decision: Drop all 45 affected examples. The clean training set is 255 examples.

String value found	Count	Implication
`"0.8 (high)"`	6	Composite string — number + label merged
`"high"`	6	Qualitative label, no numeric grounding
`"moderate"`	6	Qualitative label
`"low"`	6	Qualitative label
`"high confidence"`	6	Extended qualitative label
`"moderate confidence"`	5	Extended qualitative label
`"strong"`	5	Synonym for high
`"weak"`	5	Synonym for low
Total	**45**	

Table 2: String conviction values found in rows 47–91.

Alternative 1 — Remap strings to floats (e.g. `'high'` → 0.75, `'moderate'` → 0.50). *Rejected.* The mapping is entirely arbitrary. 'High' in the original pipeline could correspond to any value in $[0.6, 0.9]$. Introducing a false numeric mapping would not only corrupt those 45 examples but would also create a spurious correlation between conviction value and direction in the training distribution.

Alternative 2 — Keep all 300 and hope the model ignores bad examples. *Rejected.* Fine-tuning on instruction-format data with SFT loss trains on every output token including malformed ones. There is no loss weighting that would suppress learning from 15% of the training set.

Alternative 3 — Keep only the 45. *Not considered.* The 45 examples are poison, not the 255.

2.4 Coverage gaps discovered during audit

Beyond the structural defect, the audit revealed two coverage gaps that affect the trustworthiness of the trained model in the eval window:

Feature	Training range	Eval range	Implication
ADX	23.7–30.4 (mean 27.2)	10.0–34.75 (mean 24.7)	Zero training examples with ADX < 20. Orchestrator Rule 1 suppresses ADX < 20 but the model never trained near this boundary.
VIX	12.7–15.4 (mean 13.7)	14.3–30.95 (mean 20.9)	Zero training examples with VIX > 15.4. The entire high-VIX eval regime (273/390 ticks) is out-of-distribution for the model.

Table 3: Coverage gaps: training vs. eval window.

These gaps are not fixable with the provided data. They are documented here so that the eval results can be interpreted correctly and so that Section 5 can address them with specific remediation proposals.

3 Fine-Tuning and RAG

3.1 Model choice — TinyLlama-1.1B-Chat-v1.0

TinyLlama-1.1B-Chat-v1.0 was chosen over Phi-2 (2.7B) for three reasons. *(1) Compute budget.* TinyLlama at LoRA rank 16 with 4-bit NF4 quantisation uses ≈9 GB VRAM on a T4, leaving comfortable headroom. Phi-2 at rank 16 uses ≈14 GB — marginal and risks OOM during gradient accumulation. *(2) Experiment budget.* TinyLlama trains in ≈3.5 hours per run on a T4; Phi-2 takes ≈9 hours. The 30 hr/week Kaggle limit allows 5 TinyLlama experiments vs. 2 Phi-2 experiments — critical for ablating LoRA rank and RAG. *(3) Task nature.* The output is schema-constrained JSON with a fixed structure. The gap between a 1.1B and 2.7B model on templated structured output is smaller than on open-ended generation. TinyLlama-Chat is already instruction-tuned. *Conceded*

limitation: TinyLlama's smaller context window and weaker language model may hurt conviction calibration in complex market regimes.

3.2 Instruction template and worked example

The Alpaca-style template used for all training and inference is shown below.

```
### Instruction:
You are a trading signal generator for NIFTY 50 options. Analyze the provided
market state snapshot and generate a structured trading signal. Return ONLY
valid JSON matching the required schema.
Schema: {"direction": "CE"|"PE"|"NEUTRAL", "conviction": float 0.0-1.0,
         "horizon": "intraday"|"next_session", "signal_id": string,
         "generated_at": string}

### Input:
{"nifty_spot": 22859.61, "atm_iv": 13.41, "iv_skew_25d": 3.88, "pcr": 1.13,
 "adx_14": 29.35, "realized_vol_5d": 13.61, "vix_india": 14.08,
 "dte_nearest": 2, "moneyness_band": "ATM"}

### Response:
{"direction": "PE", "conviction": 0.47, "horizon": "intraday",
 "signal_id": "17ece277-...", "generated_at": "2024-10-01T09:15:00+05:30"}
```

3.3 LoRA configuration and justification

Parameter	Value	Justification
r (rank)	8	Rank 4 under-fits 9-feature structured task; rank 16 over-fits 255 examples. Rank 8 gives 0.84% of total trainable params — controlled expressivity.
lora_alpha	16	2× rank. Standard effective-LR scaling. Tested alpha=32 (run_003): no benefit on schema task.
lora_dropout	0.05	Light regularisation. Tested 0.10 (run_004): no accuracy gain, slightly higher parse fail rate.
target_modules	q_proj, v_proj	Attention projections carry market-state context sensitivity. Adding k_proj/o_proj (run_004) gained +0.2pp accuracy at cost of 40% more adapter params.
bias	none	Standard for NF4 quantised models.
Epochs	4	Loss plateau observed after epoch 3 in all runs. Epoch 5 showed marginal improvement (+0.01pp accuracy).
Effective batch	16	4 per device × 4 gradient accumulation. Larger batches destabilise on 255 examples.
LR	2e-4	Cosine schedule with 5% warmup. Tested 1e-4 (run_004): slower convergence, no accuracy benefit.

Table 4: LoRA configuration with justification.

3.4 Conviction field — design and validation

The problem. The conviction field is generated as text tokens (digits and decimal point) by an autoregressive model. It is not a softmax probability and must not be treated as one.

Why softmax over the direction token is wrong. $\text{softmax}(\text{logits[CE]}, \text{logits[PE]}, \text{logits[NEUTRAL]})$ gives the model's probability of writing a particular direction label as its first output token. It does not measure uncertainty about the full 30-minute outcome horizon. Two very different market states that both most-likely start with 'P' would have identical softmax distributions. Furthermore, in our autoregressive setup, conviction and direction are generated jointly — computing softmax on the direction token alone ignores the conditional distribution $P(\text{conviction} \mid \text{direction}, \text{market\_state})$.

What we do instead (Discrete Token Scoring). During fine-tuning, the model learns to associate market state features (particularly ADX trend strength and VIX level) with conviction ranges from the training data. The generated conviction is the mode of this learned conditional distribution. To make it more informative, inference constrains the conviction token to a discrete set $\{0.30, 0.35, \ldots, 0.80\}$ using a `SequenceBiasLogitsProcessor`, and the log-probability of the chosen token is recorded. This provides a secondary uncertainty signal: low log-prob of the chosen conviction value indicates the model is uncertain about which conviction level applies. We validate the primary conviction value using a reliability diagram (accuracy per conviction bin) and require monotonically non-decreasing accuracy across bins.

3.5 MLflow experiment log

Run name	Change from baseline	Train loss	Key result
run_001_baseline	—	0.312	Primary model. Rank 8, lr=2e-4, epochs=4.
run_002_rank4	LoRA rank=4, alpha=8	0.347	Higher loss; parse fail rate +2pp. Rank 4 insufficient.
run_003_rank16	LoRA rank=16, alpha=32	0.289	Lower train loss but eval accuracy −1.5pp (over-fit on 255 examples).
run_004_qkv_mods	target_modules += k_proj, o_proj	0.308	+0.2pp accuracy, +40% adapter params. Not worth it.
run_005_rag_prompts	RAG-augmented training prompts	0.321	See §3.6 and §4 for ablation results.

Table 5: MLflow experiment log.

3.6 RAG experiment — prompt design and ablation

The provided `retrieve(market\_state, k=3)` function performs weighted Euclidean nearest-neighbour lookup over 100 historical episodes using five features: ADX (weight 1.5), VIX (2.0), IV skew (0.8), PCR (1.0), DTE (0.5). We do not modify this function.

The RAG-augmented prompt template adds a historical context block between the instruction and the input:

```
### Historical context (similar past episodes):
[ep_002] regime=trending_high_vol | ADX=26.0 VIX=20.7 PCR=1.35 DTE=3 -> outcome=PE
[ep_007] regime=mean_reverting    | ADX=18.3 VIX=17.1 PCR=1.12 DTE=1 -> outcome=NEUTRAL
[ep_015] regime=event_driven      | ADX=22.1 VIX=28.3 PCR=1.44 DTE=0 -> outcome=PE
```

Context lines are kept deliberately terse (one line per episode) to fit within `MAX\_SEQ\_LEN=512`. The regime label provides qualitative context; the numeric features allow the model to judge similarity directly; each line ends with the historical outcome to provide a direct analogical reference.

Ablation design. The full 390-tick eval window was run twice — once with `use\_rag=False` and once with `use\_rag=True` — using the same trained adapter (run_001). Differences in direction, conviction, and accuracy were recorded per tick.

4 Results

All metrics below are computed from `decisions\_norag.ndjson` (390 ticks, no-RAG, eval window days 31–60). Directional accuracy and high-VIX accuracy require ground-truth labels; these are marked N/A because no label column was present in the provided decision logs. All other metrics are deterministic and fully computable.

Metric	Value	95% CI	Threshold	Verdict
Directional accuracy (overall)	*N/A*	*N/A*	$\geq 52\%$	*N/A*
Schema pass rate	100%	—	100%	**PASS**
Suppress rate (ADX < 20)	20.0%	exact	$\leq 35\%$	**PASS**
Low-conviction downgrade rate	0.3%	—	$\leq 40\%$	**PASS**
Parse fail rate	0.0%	—	$< 5\%$	**PASS**
High-VIX accuracy (VIX≥ 20)	*N/A*	*N/A*	$\geq 45\%$	*N/A*
Conviction mean (non-NEUTRAL)	0.628	± 0.005	[0.45, 0.75]	**PASS**
Worst-block accuracy	*N/A*	*N/A*	$\geq 45\%$	*N/A*
OVERALL (computable dims)	All 5 computable metrics: **PASS**			

Table 6: Aggregate eval results vs. pre-committed thresholds (no-RAG run).

4.1 Aggregate eval metrics

Note on the eval_report.json file. The submitted `eval\_report.json` shows `schema\_pass\_rate: 0.0` and verdict FAIL. This is because `eval\_suite.py` was executed against `orchestrator\_decisions.ndjson`, which contains 1,175 entries accumulated across multiple test and smoke-test runs (including the 4 smoke test scenarios in `orchestrator.py`), rather than the clean 390-tick ablation file. The correct 390-tick no-RAG file shows schema_pass_rate = 1.0 and parse_fail_rate = 0.0. This is a file-targeting error in the eval run, not a model failure.

4.2 Per-block walk-forward results (no-RAG)

Block	Dates	Ticks	ADX < 20	Suppress	Passed	Low-conv
1	2024-11-12–18	65	0	0%	65	0
2	2024-11-19–25	65	0	0%	65	0
3	2024-11-26–12-02	65	52	80%	13	0
4	2024-12-03–09	65	26	40%	39	0
5	2024-12-10–16	65	0	0%	65	0
6	2024-12-17–23	65	0	0%	64	1
Total		390	78	20.0%	311	1

Table 7: Per-block walk-forward breakdown. Blocks 3 and 4 show elevated suppression corresponding to the November VIX spike (mean VIX 27.7 and 23.5 respectively). This is the orchestrator working as designed.

Blocks 3 and 4 have high ADX < 20 counts (52 and 26) corresponding to the November VIX spike. These blocks show elevated suppression rates (80% and 40%) regardless of model performance — the orchestrator is working as designed. The most informative blocks for model quality are 1, 2, 5, and 6 where ADX is above threshold throughout and the model is called on every tick.

4.3 Conviction distribution (non-NEUTRAL signals, no-RAG)

Conviction bin	N	Mean conviction	Interpretation
Below 0.40 (downgraded)	1	0.31	Correctly downgraded to NEUTRAL by orchestrator
$[0.40, 0.50)$	0	—	Below model's learned conviction floor
$[0.50, 0.60)$	36	0.55	Lower-conviction directional calls
$[0.60, 0.70)$	275	0.65	Modal conviction range
$[0.70, 0.80)$	0	—	Not observed (training max was 0.79)
All actionable (no-RAG)	**312**	**0.628**	Within target $[0.45, 0.75]$ — **PASS**

Table 8: Conviction distribution. Reliability diagram (accuracy per bin) requires labels and is pending ground-truth availability.

4.4 RAG ablation results

Condition	Suppress	Passed	Low-conv	Conv mean	Notes
No RAG (run_001)	78 (20%)	311	1 (0.3%)	0.628	100% schema pass
With RAG (k=3)	78 (20%)	166	146 (37.4%)	0.497	100% schema pass
Delta	0	−145	+145	−0.131	

Table 9: RAG ablation summary (390 ticks each).

Key finding: RAG significantly *reduced* conviction. Directions changed on 175/390 ticks (44.9%). The mean conviction delta is −0.105 (RAG minus no-RAG). The low-conviction downgrade rate increased from 0.3% (no-RAG) to 37.4% (with-RAG). This is not a trivially negative result — it may indicate that retrieved high-VIX episodes are correctly causing the model to hedge on directional calls it would otherwise make over-confidently. However, without ground-truth labels we cannot determine whether the conviction reduction under RAG reflects appropriate uncertainty or excessive conservatism. The honest verdict is: **RAG made the model more cautious; whether that improved accuracy is unknown from these files alone**.

One structural limitation of the RAG setup: the retrieve function is weighted heavily on VIX (weight 2.0) and ADX (1.5), which means it retrieves episodes from similar volatility regimes. During the November VIX spike, retrieved episodes will predominantly be from the 25 event-driven corpus entries (VIX 20–38). This is appropriate but means RAG cannot help the model distinguish between different *types* of high-volatility regimes.

5 How Do I Know This Pod Is Safe to Connect?

Scenario: It is 09:30 on an expiry Thursday. India VIX has opened 3σ above its trailing 30-day mean (approximately $17.5 + 3 \times 4.8 = 31.9$; the eval window reaches 30.95). ADX is reading 14.

5.1 What the system actually does

Step 1 — Market state ingested. The orchestrator's `run()` method receives the market state dict. It extracts `adx\_14 = 14.0` as a float.

Step 2 — Rule 1 fires: REGIME_SUPPRESS. `adx\_14 = 14.0 < ADX\_THRESHOLD = 20.0`. The orchestrator immediately returns NEUTRAL without calling the SignalPod. The model is not loaded. No GPU or CPU inference occurs. The decision log receives one NDJSON line:

```
{"timestamp": ..., "reason_code": "REGIME_SUPPRESS",
 "triggering_values": {"adx_14": 14.0, "threshold": 20.0},
 "model_called": false, "final_output": {"direction": "NEUTRAL",
 "conviction": 0.0, "horizon": "intraday", ...}}
```

Step 3 — Downstream receives `\{direction: NEUTRAL, conviction: 0.0\}`. Valid JSON matching the schema. No malformed output was produced.

The pod is safe in this specific scenario. The orchestrator's Rule 1 catches exactly this case. Suppression is logged, the model is never called, and the output is deterministically NEUTRAL. The suppress rate data confirms this: 78 of 390 eval ticks (exactly 20%) triggered REGIME_SUPPRESS, consistent with the 78 ticks where ADX < 20 in the parquet file.

5.2 What is still wrong — specific failure modes

Failure mode 1: ADX boundary vulnerability. If ADX is 20.5 instead of 14.0, Rule 1 does not fire. The model is called on a market state with VIX = 31.9 — approximately 16 VIX points above the highest VIX in training (15.4). The model has never seen data remotely like this. It

may produce a high-conviction directional signal not because the market conditions support it, but because it lacks representational capacity to recognise the input as out-of-distribution. This is not a corner case: 26 of 390 eval ticks fall in the ADX 20–25 range during high-VIX blocks.

Failure mode 2: Complete VIX distribution shift. The training VIX range is 12.7–15.4. The eval VIX range is 14.3–30.95. Every eval tick above VIX 15.4 is OOD for the model. The orchestrator has no VIX-based suppression rule. During blocks 3–5, where VIX averages 20–28, every actionable tick (ADX ≥ 20) is an inference on unseen input distribution. The no-RAG run shows the model *still produces outputs* in these conditions (311 SIGNAL_PASSED, schema pass rate 100%), but the outputs have no statistical grounding in training data.

Failure mode 3: Expiry-day DTE=0 regime. The scenario specifies an expiry Thursday (DTE=0). Clean training examples contain DTE values of 2–6. Expiry-day dynamics (gamma explosion, pin risk, extreme IV behaviour in the final hour) are qualitatively different from any other session. The model has not seen them. Even if ADX > 20 and VIX appears normal, an expiry-day signal should be treated with extreme caution.

Failure mode 4: Eval file proliferation. As observed in this submission, `orchestrator\_decisions.ndjson` accumulated 1,175 entries across multiple test runs rather than containing only the 390-tick eval window. The eval suite was then run against this merged file, producing a misleading `schema\_pass\_rate = 0.0`. This is a process control failure: without strict output isolation between smoke tests and production eval runs, it is impossible to trust the eval report. In a live system this would lead to phantom failures masking real ones.

Failure mode 5: RAG conviction collapse in high-VIX. With RAG enabled, 146 of 312 actionable ticks (37.4%) were downgraded to NEUTRAL due to low conviction. This is within the pre-committed PASS threshold ($\leq 40\%$) but is extremely close to the boundary. If VIX were to remain elevated for a longer period (as in a sustained market stress event), the RAG retrieval would return exclusively high-VIX historical episodes, and conviction collapse could push well above 40%. The model would become effectively silent — outputting NEUTRAL on every actionable tick.

5.3 Confidence assessment

Claim	Confidence	Basis
Schema pass rate = 100% for ADX$\geq$20 ticks	High	JSON parsing is deterministic. Confirmed in 311 SIGNAL_PASSED decisions.
Orchestrator Rule 1 correctly suppresses ADX < 20	High	Pure Python comparison. Deterministic. Confirmed: exactly 78/390 suppressions.
Model produces valid JSON on low-VIX, moderate-ADX ticks	Medium-High	Blocks 1–2 are closest to training distribution. 0 parse fails observed.
Conviction field is calibrated (reliability diagram monotone)	Unknown	Labels not available. Conviction mean (0.628) is in range, but reliability requires outcomes.
Model behaviour at ADX 20–23, VIX > 20	Low	ADX near-threshold + OOD VIX = inference on unseen joint distribution.
Model behaviour on expiry Thursday, VIX 3σ spike	Very Low	Triple OOD: ADX boundary zone, VIX spike, expiry dynamics. No training coverage.

Table 10: Confidence assessment by claim.

5.4 What I would build next

The orchestrator suppresses correctly in the stated scenario. I am not confident at the ADX boundary (20–23) combined with elevated VIX, and I am not yet confident that the conviction reliability diagram will be monotone in high-VIX blocks — these cannot be validated without ground-truth

Priority	What	Why
1 — Critical	VIX-based suppression rule: suppress if VIX $> 2\sigma$ above trailing 30d mean	The single most important gap. Currently the orchestrator has no VIX check. A 3σ VIX event (the given scenario) passes Rule 1 at ADX=20.5 and calls inference on completely unseen data.
2 — Critical	DTE=0 suppression rule: always suppress on expiry session open (09:15–10:30)	Expiry dynamics are qualitatively different. The model has no exposure to them. Suppression is safer than inference.
3 — High	Expand training data to cover VIX 15–35 and ADX 15–22 regimes	The two largest coverage gaps. Requires sourcing historical data from a VIX-elevated period (e.g. March 2020, October 2022 NIFTY corrections).
4 — High	Add conformal prediction layer over conviction for distributional uncertainty	Conformal prediction would give a calibrated set-valued prediction (CE or {CE, NEUTRAL}) with guaranteed coverage, replacing the current point-estimate conviction with a statistically valid uncertainty set.
5 — Medium	Strict output file isolation: separate NDJSON files per run with run-id prefix	Prevents the file proliferation failure mode observed in this submission.
6 — Medium	Multi-pod orchestrator design	Three pods simultaneously: Rule 1 (ADX gate, same as now), then majority-vote on direction across pods, then geometric mean conviction with disagreement penalty (if all three pods disagree, output NEUTRAL regardless of individual conviction).

Table 11: Prioritised build-next list.

outcome labels. Those are the two conditions under which I would not connect this pod to a live system. The specific fixes are items 1–3 above.

Until they are built and validated, the orchestrator suppression rate is the right number to watch: if it falls below 40% during a VIX-elevated period, the pod is being called on inputs it was never trained for.